

Sistema de Navegacion Autonoma

Robot Ackermann con Vision por Computadora

Documentacion Tecnica del Proyecto
Marzo 2026

1. Introduccion

Este proyecto implementa un sistema completo de navegacion autonoma para un robot con cinematica Ackermann (tipo automovil). El sistema integra vision por computadora, planificacion de trayectorias, algoritmos de seguimiento de caminos y comunicacion inalambrica para controlar un robot fisico basado en ESP32.

El robot opera dentro de un espacio de trabajo observado por una camara cenital (vista superior). Un sistema de procesamiento en PC detecta la posicion y orientacion del robot mediante marcadores de color, calcula la trayectoria optima hacia un objetivo y envia comandos de control al robot a traves del protocolo MQTT a una frecuencia de 30 Hz.

2. Arquitectura del Sistema

El sistema sigue una arquitectura modular organizada en cinco componentes principales que forman un ciclo de control cerrado:

Modulo	Funcion Principal	Tecnologia
vision/	Captura de camara y deteccion del robot	OpenCV, NumPy
navigation/	Generacion de objetivos y trayectorias	Bezier, Pure Pursuit
control/	Maquina de estados y conversion a comandos	Python
communication/	Comunicacion MQTT con ESP32	paho-mqtt
ui/	Interfaz grafica con visualizacion en tiempo real	PyQt5
esp32/	Firmware del robot (motor y servo)	Arduino/C++

Flujo de Control (30 Hz)

1) La camara captura un frame. 2) El detector identifica los marcadores verde (frente) y azul (trasero) del robot mediante segmentacion HSV, obteniendo posicion (x, y) y orientacion (theta). 3) El controlador ejecuta el algoritmo Pure Pursuit sobre la trayectoria Bezier planificada. 4) Se calculan los comandos de actuacion (PWM del motor y angulo del servo). 5) Los comandos se envian por MQTT al ESP32 en formato JSON. 6) La interfaz grafica muestra la visualizacion en tiempo real.

3. Modulo de Vision

La deteccion del robot se realiza mediante segmentacion por color en el espacio HSV. El robot porta dos marcadores de al menos 3x3 cm:

Marcador	Color	Rango H	Rango S	Rango V
Frontal	Verde	35-85	80-255	80-255
Trasero	Azul	100-130	80-255	80-255

El proceso incluye: conversion BGR a HSV, creacion de mascarar binarias, operaciones morfologicas de apertura y cierre para eliminar ruido, deteccion del contorno de mayor area para cada color, y calculo de centroides mediante momentos de imagen. La posicion del robot es el punto medio entre ambos centroides, y la orientacion se obtiene con atan2 del vector frontal-trasero.

4. Navegacion y Planificacion de Trayectorias

4.1 Generacion de Objetivos

El usuario define un vector de movimiento (distancia en pixeles y angulo en grados) desde la interfaz grafica. El modulo TargetGenerator proyecta este vector desde la pose actual del robot para calcular la posicion y orientacion objetivo.

4.2 Trayectorias Bezier Cubicas

Se genera una curva Bezier cubica con 80 puntos interpolados. Los puntos de control se calculan como: P0=inicio, P1=inicio + (d/3)*direccion_inicio, P2=meta - (d/3)*direccion_meta, P3=meta. Esta parametrizacion garantiza trayectorias suaves compatibles con las restricciones no-holonomicas del modelo Ackermann.

4.3 Algoritmo Pure Pursuit

El seguimiento de trayectoria utiliza el algoritmo Pure Pursuit, un metodo clasico y robusto para robots con cinematica tipo automovil. El proceso es: 1) Encontrar el punto mas cercano de la trayectoria al robot. 2) Buscar el punto de lookahead a una distancia configurable (30 px por defecto). 3) Transformar el punto al marco local del robot. 4) Calcular la curvatura: $k = 2 \cdot y_{\text{local}} / Ld^2$. 5) Obtener el angulo de direccion Ackermann: $\delta = \arctan(L \cdot k)$, donde L es la distancia entre ejes (wheelbase = 50 px).

La velocidad se adapta inversamente al angulo de direccion: a mayor giro, menor velocidad (rango 20%-100% del PWM maximo), mejorando la estabilidad en curvas.

5. Sistema de Control

5.1 Maquina de Estados

Estado	Descripcion	Transicion
IDLE	Sin navegacion activa	-> NAVIGATING al recibir objetivo
NAVIGATING	Siguiendo trayectoria	-> REACHED si dist < 15 px
REACHED	Objetivo alcanzado	-> IDLE (automatico)
ERROR	Robot no detectado	-> NAVIGATING al redetectar

5.2 Conversion a Comandos de Actuacion

El controlador convierte las salidas del Pure Pursuit en senales para el hardware: el angulo del servo se calcula como $90 - \text{angulo_direccion}$ (centrado en 90, rango 45-135 grados). El PWM del motor es proporcional a la velocidad calculada. Se aplica control diferencial para compensar el deslizamiento en curvas: $\text{PWM}_{\text{izq}} = \text{PWM} \cdot (1 + 0.3 \cdot \text{factor_giro})$, $\text{PWM}_{\text{der}} = \text{PWM} \cdot (1 - 0.3 \cdot \text{factor_giro})$.

6. Comunicacion MQTT

La comunicacion entre el PC y el ESP32 se realiza mediante el protocolo MQTT a traves de un broker Mosquitto. Se utilizan dos topicos:

Topico	Direccion	Contenido (JSON)
robot/cmd	PC -> ESP32	left_pwm, right_pwm, servo, action
robot/status	ESP32 -> PC	status, wifi_rssi, uptime_s

El ESP32 publica su estado cada 2 segundos e implementa reconexion automatica. La accion puede ser "move" (movimiento) o "stop" (parada de emergencia). La comunicacion es thread-safe con locks para acceso concurrente.

7. Configuracion de Hardware

7.1 Componentes del Robot

- Microcontrolador: ESP32 con WiFi integrado
- Motor DC de traccion controlado por puente H (L298N)
- Servo para direccion (cinematica Ackermann)
- Marcadores de color: verde (frente) y azul (trasero), minimo 3x3 cm

7.2 Configuracion de Pines ESP32

Componente	Pin GPIO	Funcion
Motor ENA	25	PWM de traccion
Motor IN1	26	Direccion del motor
Motor IN2	27	Direccion del motor
Servo	13/15	Control de direccion

7.3 Infraestructura

- Camara USB en posicion cenital sobre el area de trabajo (640x480 px)
- Red WiFi local conectando PC y ESP32
- Broker MQTT Mosquitto (puerto 1883) ejecutandose en el PC o en la red

8. Interfaz Grafica de Usuario

La aplicacion PyQt5 presenta un diseno de dos paneles: el panel izquierdo muestra el video en vivo (640x480) con superposiciones

de la posicion del robot (circulo cyan), trayectoria planificada (linea naranja), objetivo (cruz roja), punto de lookahead (punto magenta) y cuadrícula de coordenadas. El panel derecho contiene controles agrupados: estado del robot (X, Y, theta), vector de movimiento (distancia y angulo con botones Enviar/Detener), informacion de navegacion (distancia al objetivo, error de seguimiento), y configuracion MQTT (IP del broker, puerto, estado de conexion).

9. Modos de Ejecucion

Comando	Modo	Uso
python main.py	Estandar	Camara por defecto, broker local
python main.py --camera 1	Camara alternativa	Seleccionar dispositivo
python main.py --image foto.png	Imagen estatica	Pruebas de deteccion
python main.py --simulate	Simulacion	Sin camara conectada
python main.py --broker IP	Broker remoto	ESP32 en otra red

10. Stack Tecnologico

Tecnologia	Version	Proposito
Python	3.8+	Lenguaje principal del sistema
OpenCV	>= 4.8.0	Procesamiento de imagen y vision
NumPy	>= 1.24.0	Calculos numericos y vectoriales
PyQt5	>= 5.15.0	Interfaz grafica de usuario
paho-mqtt	>= 2.0.0	Protocolo de comunicacion MQTT
Matplotlib	>= 3.7.0	Visualizacion auxiliar
Arduino/C++	-	Firmware del ESP32
PubSubClient	-	Cliente MQTT en ESP32